Teaching Statement

Brent Pappas
University of Central Florida, Department of Computer Science
brent.pappas@ucf.edu | www.pappasbrent.com

My teaching and mentoring prepares students to develop and research software by *learning their tools*. This not only includes learning about concrete tools such as command line utilities and programming languages, but also conceptual tools such as algorithms, software design strategies, and defensive programming tactics. This approach to instruction teaches students to use all facets of every utility available to them when solving problems, empowering them to not only create robust software, but to do so with alacrity. This produces a sense of mastery, initiating a positive feedback loop in which students perpetually seek opportunities to expand their knowledge further. To provide students with such opportunities, I recruit them to conduct research with me on software engineering and security, giving them the chance to hone their skills while contributing to the broader field of Computer Science.

Summary of Teaching Experience

While pursuing my PhD at the University of Central Florida (UCF), I was the instructor of record for **COP 3402: Systems Software** for both the Spring 2026 and Summer 2026 semesters. I am currently teaching the Fall 2026 semester of the course. Systems Software is a required course for UCF computer science majors with consistently high enrollment numbers: my Spring 2026 section had 243 enrolled students, my Summer 2026 section had 107, and the Fall 2026 section I currently teach has 210 enrollments. Most students choose to take Systems Software during their sophomore year, after completing prerequisite courses on C programming, and data structures and algorithms. As such, Systems Software is usually the first course students take that requires them to use industry-standard software development tools such as Git and the Linux command line. For many students learning how to use these tools can be difficult, and so to help my class succeed, I spend the first half of each semester teaching students the basics of developing software on Linux, and then use these concepts in the latter half of the course to help teach the basics of compiler design.

I began teaching the course with a curriculum prepared by my advisor, and I have since made the course my own by incorporating student feedback. For instance, some students in my Spring 2026 section stated that they felt that the course lecture notes were too austere, and so in response I have expanded the course web pages to contain more than just speaker notes, but rather complete passages explaining the concepts and examples reviewed in lecture. Additionally, some students in my Spring 2026 section wished for lectures to explain fundamental concepts more thoroughly, and to fulfill this request I have created new illustrated examples which I work through in lecture to explain concepts such as Linux pipes, redirection, and x86 64 assembly calling conventions. Furthermore, I record all lectures and post the recordings after class for students to review. To incentivize students to still attend class and not only watch the recordings, I make attendance mandatory (but only worth 4% of the overall grade). Across all my semesters teaching, I have achieved high course (4.8 / 5 average vs. 4.34 department average) and instructor (4.67 / 5 average vs. 4.27 department average) evaluation ratings.

Before teaching COP 3402, I served as a GTA for the course, with my advisor being the instructor, during both the Spring and Fall of 2025. During these semesters I led small (30 students or fewer) labs, in which I would answer student questions, review project assignments, and provide enriching examples of real-world applications of course content. The effectiveness of these techniques is evinced by the glowing reviews left by students, not only for myself (4.93 / 5), but also for my advisor, who, with my assistance as a GTA, earned the highest ratings he had ever received for the course's structure (4.92 / 5) and instruction (4.89 / 5).

Next semester I am set to teach **COP 5621: Compiler Construction**. This is a graduate-level course focused on teaching students the six major phases of compiling source code to assembly code: lexing, parsing, semantic analysis, IR code generation, optimization, and assembly code generation. I plan to teach this course by requiring students to implement each of these phases in plain C using starter code that I will provide. Since

this is a graduate-level course, I plan to also incorporate my original research on translating C preprocessor macros to C code.

Teaching Awards

I have received GTA excellence awards from UCF at both the college and university level. This latter award was only given to one out of 800+ GTAs across all of UCF, and I was the sole recipient.

Teaching Techniques and Research Mentoring Methods

I use the following tactics to teach and mentor students. These methods come from my personal experience teaching Systems Software, and also from professional training I obtained from the UCF graduate student course **IDS 6513: Preparing Tomorrow's Faculty**, in which I learned basic pedagogical concepts such as Bloom's taxonomy.

I Work Backwards from Concrete Objectives

Before preparing any course material, I choose specific student learning outcomes. This provides a concrete course structure, and enables students to check their progress towards course goals. I also use this tactic when collaborating with undergraduate researchers, beginning new research projects by first designing research questions. This helps students remember overarching research objectives when performing research tasks.

Teaching example. To teach students about version control with Git, I worked backwards by first choosing the specific Git commands I wanted students to learn (e.g., `add`, `commit`, `push`). I then provided illustrated examples of these commands during lecture, and tested student understanding by assigning a project on using Git to create a local repository, stage and commit changes to it, and finally upload those changes to a remote repository. The class average for this project has always been an A, attesting to the efficacy of working backwards to design course material.

Mentoring example. I began working with a team of undergraduate research assistants to conduct research on securing software build phases by preparing and disseminating a list of the project's goals. I then decomposed these goals down into specific tasks, and created GitHub issues with instructions for undergraduate assistants on how to complete them. Utilizing GitHub issues had the additional benefit of enabling all members of the team to track our progress towards meeting the project's goals. After a semester of organized collaboration, this work has resulted in a publication under review at the 49th IEEE/ACM International Conference on Software Engineering.

I Leverage the Familiar

I start with familiar content before moving into unknown territory. This not only engages students, but also facilitates learning by easing students into new information. This technique also helps me on-board research assistants onto projects by first giving them small and easy tasks before assigning more complex ones.

Teaching example. To teach students how to solve software engineering problems by slowly decomposing larger problems into smaller ones, I begin with the familiar fable of the tortoise and the hare. Students already know the fable, and so the message becomes intuitive: expedience leads to failure, whereas prudence begets success. Establishing this message early eases my way in to giving students specific examples of careful programming practices.

Mentoring example. When I recruited Daniel Henriquez to assist me with my research on securing software build systems, I first assigned him a task that built off what he had learned while taking computer science courses at UCF. Specifically, I tasked him with cataloging the commands to configure, compile, and test projects using GNU Make, a tool which he had already been using to complete course projects. Since he was already familiar with Make, he was able to complete the task swiftly, and has since used his research experience to obtain an internship Bogen Communications.

I Reward Success

To incentivize students to be continuous learners, I allow students to continuously resubmit late projects for a small point penalty, and offer extra credit for implementing bonus features in programming assignments. This

approach teaches students to persevere past their failures, rather than to fear their mistakes. In a research setting, this approach encourages students to obtain novel results by exploring new ideas.

Teaching example. When teaching COP 3402, I test student ability to create systems software by tasking them with implementing their own shell in C. Many students find this task difficult, because it requires using in conjunction all the Linux system calls they will have learned up that point in the course. Consequently, many students initially score only partial points on this assignment, which in another course would be their final grade for the project. But, since I allow students to resubmit the project until the end of the semester to regain lost points, many students keep working to improve their scores. By the end of the semester, the average class grade for the project is always an A. But more importantly, continuously working on the project helps students reinforce their knowledge of the Linux shell, with the median score for shell-related questions on the final exam for COP 3402 consistently being at least 85%.

Mentoring example. When conducting research on translating C preprocessor macros to C language constructs, I gave my undergraduate assistant Joseph Zalusky great freedom in choosing how to implement a tool for performing macro-to-C translation. As his mentor, I supported Joey by supplying him with the formal rules for performing the translation, giving him code review, and providing general guidance on how to conduct research. With our combined efforts we produced the first semantically-aware macro-to-C conversion tool, MerC, which has led to an accepted paper at the 41st IEEE/ACM International Conference on Automated Software Engineering. For his hard work Joey was made a co-author on the paper, and he used his research experience to obtain a full-time software engineering position at Moog, Inc.

Summary of Mentoring Experience

While completing my PhD at the University of Central Florida, I have mentored many undergraduate students across several different research projects. In 2024 I recruited Zachary Burkett and Joseph Zalusky and to assist me with research on the semantic-aware translation of C preprocessor macros to C programming language constructs. This collaboration resulted in a conditionally-accepted paper at the 41st IEEE/ACM International Conference on Automated Software Engineering, with both undergraduate students being co-authors. Both these students leveraged their research experience to secure positions they were seeking, with Joey obtaining a full-time software engineering role at Moog Inc., and Zachary being accepted to the Masters of Sound and Music Computing program at the Pompeu Fabra University.

In 2026 I recruited a team of seven undergraduate students to assist me with research on securing software build phases against pipeline poisoning attacks: Adam Betinsky, David Gusmao, Michael Johnson, Brayden Coggin, Daniel Henriquez, Thiago Airoldi, and Rasmin Ahmed. Four of these students discovered an interest in research while taking a course for which I served in a teaching capacity, with Adam, Michael, and David being enrolled in lab sessions I led as a GTA for COP 3402 in 2025, and Rasmin being a student in the section of COP 3402 I was the instructor of record for in Spring 2026. The work this team of students has produced has led to a publication currently under review at the 49th IEEE/ACM International Conference on Software Engineering. As with my previous undergraduate collaborations, these students have used the research experience they gained while working with me to obtain competitively-sought after roles, with Adam securing an internship at Keystone Strategy, and Daniel earning an internship at Bogen Communications.

Conclusion

My teaching and mentoring center on equipping students with the skills to become proficient software engineers. I achieve this through three key techniques: **working backwards**, **leveraging the familiar**, and **rewarding success**. Using these techniques, I have achieved high course (4.8 / 5 average vs. 4.34 department average) and instructor (4.67 / 5 average vs. 4.27 department average) evaluation ratings, as well multiple awards for GTA excellence. I have also applied these three techniques to undergraduate mentoring to produce several top-tier publications with undergraduate researchers, many of whom have leveraged the research experience they gained working with me to obtain positions in industry and academia. I will continue to improve on these techniques to deliver quality instruction to my students, and enriching research opportunities to my mentees.

I am most interested in teaching courses that foster advanced competencies in software development. Specifically, I would most like to teach Systems Software, Software Engineering, and Compiler Construction. Additionally, I would also be willing to teach courses on compiler design and programming languages, since my graduate research work often overlapped with these areas. I am also qualified and capable of teaching introductory courses on computer programming, data structures, and algorithms.